project_4.pdf
PDF

can you make a outline to complete this project

Thought for 8s

Here's a concrete, step-by-step outline you can follow to complete this project from start to finish. project_4

0. Setup & Planning

1. **Create a project folder**
   - `project4/`
     - `kmeans_em.py` (main code)
     - `plots/` (save all figures)
     - `results/` (optional: CSVs / NumPy pickles)
     - `report/` (for your final PDF)
     - `ai_transcript.txt` (for AI usage summary)
2. **Set up environment**
   - Install/import: `numpy`, `matplotlib`, `seaborn`, `sklearn`.
   - Set a random seed for reproducibility.
3. **Decide datasets and parameters**
   - Datasets:
     - `make_blobs` (100–200 points)
     - `make_moons` (100–200 points)
   - `k` values: {2, 3, 4, 5}
   - K-Means init: `["random", "k-means++"]`
   - GMM covariance types: `["full", "diag"]`

1. Data Generation Module

1. **Write a function** `generate_datasets()`
   - Returns a dict like:

```python
{
  "blobs": (X_blobs, y_blobs_true),
  "moons": (X_moons, y_moons_true)
}
```

   - Use:
     - `make_blobs(n_samples=150, centers=3, cluster_std=1.0, random_state=...)`
     - `make_moons(n_samples=150, noise=0.05, random_state=...)`
2. **Quick sanity plots**
   - Scatter plot the raw `blobs` and `moons` datasets just to verify they look reasonable.

2. Part 1 – K-Means Clustering

2.1 Implementation & Running Experiments

1. **Write a function** `run_kmeans(X, k_values, inits)`
   - For each `k` in `{2,3,4,5}`
   - For each `init` in `["random", "k-means++"]`:
     - Fit `KMeans(n_clusters=k, init=init, random_state=...)`
     - Save:
       - `labels_`
       - `cluster_centers_`
       - `inertia_` (for elbow plot / ICSSD)
       - `silhouette_score(X, labels)`
2. **Call this for both datasets**
   - `kmeans_results["blobs"]`
   - `kmeans_results["moons"]`

2.2 Visualization

3. **Cluster scatter plots**
   - For each dataset & each `(k, init)` combination:
     - 2D scatter plot of data points colored by cluster label.
     - Overlay cluster centers with a distinct marker (e.g., `X`).
     - Title example:
       - `"KMeans on blobs - k=3, init=k-means++"`
4. **Optional elbow plots**
   - For each dataset:
     - Plot `k` on x-axis vs. `inertia_` (ICSSD) on y-axis for one init (e.g., `k-means++`).
     - Label clearly (axes, legend if needed).

2.3 Quantitative Evaluation Summary

2.3 Quantitative Evaluation Summary

5. **Create a small results table** (for report later)
   - Columns: `Dataset`, `k`, `init`, `Silhouette`, `Inertia`.
   - You can store in a list of dicts or a pandas DataFrame.

2.4 Discussion Notes (for report)

As you look at results, jot down bullet notes:

- How does **init** (`random` vs `k-means++`) affect:
  - Convergence speed (iterations) – can be seen from `n_iter_`.
  - Final silhouette score / inertia.
- Which dataset is **better suited for K-Means** and why?
  - Expect: `make_blobs` (spherical, well-separated) works better.
  - `make_moons` shows K-Means' difficulty with non-convex shapes.
- How does **k** affect:
  - Silhouette scores?
  - Visual cluster quality (too few vs too many clusters)?

These notes become your Part 1 discussion section.

3. Part 2 – Expectation-Maximization (Gaussian Mixture Models)

3.1 Implementation & Running Experiments

1. **Write a function** `run_gmm(X, k_values, cov_types)`
   - For each `k` in `{2,3,4,5}`
   - For each `cov_type` in `["full", "diag"]`:
     - Fit `GaussianMixture(n_components=k, covariance_type=cov_type, random_state=...)`.
     - Get:
       - Cluster labels: `gm.predict(X)`
       - Per-sample log-likelihood: `gm.score(X)`
         - Save average log-likelihood (that's what `.score` returns).
       - BIC: `gm.bic(X)`
       - Silhouette score using predicted labels.
2. **Call this for both datasets**
   - `gmm_results["blobs"]`
   - `gmm_results["moons"]`

3.2 Visualization

3. **Cluster scatter plots (GMM)**
   - For each dataset & `(k, cov_type)`:
     - Color points by predicted cluster label.
     - Optionally show:
       - Ellipses for Gaussian components (using `matplotlib.patches.Ellipse`), **or**
       - A density contour overlaid using `seaborn.kdeplot`.
     - Title example:
       - `"GMM on blobs - k=3, cov_type=full"`
4. **Visual comparison vs K-Means**
   - For a couple of representative settings (e.g., `k=2` and `k=3`), put K-Means and GMM plots side-by-side in your report for each dataset.

3.3 Quantitative Evaluation Summary

5. **Create results tables** for GMM
   - Table 1 (per dataset):
     - Columns: `k`, `cov_type`, `Silhouette`, `Avg Log-Likelihood`, `BIC`.
   - Note how:
     - Silhouette changes with `k` and `cov_type`.
     - BIC changes with `k` (typically, look for the minimum BIC to pick best k).

3.4 Discussion Notes (for report)

As you inspect GMM results, note:

- How **GMM handles elliptical / non-spherical clusters**:
  - For `make_moons`, does `full` covariance capture curved structures better than `diag`?
- In what scenarios does **EM (GMM) outperform K-Means**?
  - Overlapping clusters, non-spherical clusters, probabilistic interpretation.
- How does **covariance type** influence:
  - Cluster shapes (axis-aligned vs general ellipses).
  - Separation and silhouette scores.
- How does **BIC** guide the choice of `k` compared to silhouette?

4. Organizing Results for the PDF Report (≤ 8 pages)

Structure your report roughly like this:

1. **Title Page / Header**
   - Course, project title, your name, date
2. **1. Introduction (½–1 page)**
   - Briefly describe unsupervised learning, clustering

4. Organizing Results for the PDF Report (≤ 8 pages)

Structure your report roughly like this:

1. **Title Page / Header**
   - Course, project title, your name, date.
2. **1. Introduction (½–1 page)**
   - Briefly describe unsupervised learning, clustering.
   - Introduce K-Means and GMM/EM at a high level.
   - State objectives of the project.
3. **2. Methods (1–2 pages)**
   - **2.1 Datasets**
     - Describe `make_blobs` and `make_moons` and why they're chosen.
   - **2.2 K-Means**
     - Short description of algorithm and key parameters (`k`, `init`).
   - **2.3 Gaussian Mixture Models (EM)**
     - Short description of EM for GMMs, components, covariance types.
   - **2.4 Evaluation Metrics**
     - Silhouette score, inertia (for K-Means), log-likelihood, BIC.
4. **3. Results – K-Means (2–3 pages)**
   - **3.1 Tables**
     - Table of silhouette scores (and possibly inertia) for all `k` and `init` on both datasets.
   - **3.2 Plots**
     - Cluster scatter plots for key settings.
     - Optional elbow plots.
   - **3.3 Discussion**
     - Effect of `k`, effect of init, dataset suitability.
5. **4. Results – EM / GMM (2–3 pages)**
   - **4.1 Tables**
     - Silhouette, log-likelihood, BIC for all `k` and covariance types.
   - **4.2 Plots**
     - Cluster visualizations showing ellipses / contours.
   - **4.3 Discussion**
     - Effect of covariance type.
     - Comparison to K-Means on each dataset.
     - How BIC helps choose `k`.
6. **5. Overall Comparison & Conclusion (½–1 page)**
   - Summarize:
     - When K-Means works well vs poorly.
     - When GMM/EM is preferable.
     - Key takeaways about cluster shapes, evaluation metrics, and model selection.
7. **References**
   - Cite scikit-learn, matplotlib, seaborn docs.

5. AI Transcript

1. **Collect chat snippets / prompt summaries**
   - Either paste verbatim key prompts & responses or summarize how AI was used (e.g., "used AI to outline project steps and clarify evaluation metrics").
2. **Save as `ai_transcript.txt` or include as an appendix in the report**, depending on your instructor's preference.

6. Code Cleanup & Final Packaging

1. **Refactor your code**
   - Clear function names (`generate_datasets`, `run_kmeans`, `run_gmm`, `plot_clusters`, etc.).
   - Add comments explaining each major step.
2. **Check reproducibility**
   - Fixed `random_state`.
   - Rerun everything once to regenerate plots and tables.
3. **Export report to PDF**
   - Make sure all figures and tables are clearly labeled and referenced in text.
4. **Create final zip**
   - Contains:
     - PDF report (≤ 8 pages)
     - All Python code
     - AI transcript file
     - (Optionally) plots directory

If you'd like, next I can help you write a skeleton `kmeans_em.py` file with all the functions and loops stubs laid out so you can just fill in the details.

can you show me how you would plot the blobs and moons with matplotlib

Here's a simple example of how I'd generate and plot **blobs** and **moons** using `matplotlib` and `sklearn.datasets`.

python Copy code

```python
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_moons

# 1. Generate the data
X_blobs, y_blobs = make_blobs(
    n_samples=150,
    centers=3,
    cluster_std=1.0,
    random_state=42
)

X_moons, y_moons = make_moons(
    n_samples=150,
    noise=0.05,
    random_state=42
)

# 2. Create side-by-side plots
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# ---- Blobs ----
axes[0].scatter(
    X_blobs[:, 0],
    X_blobs[:, 1],
    c=y_blobs,
    cmap="viridis",
    s=30,
    edgecolor="k"
)
axes[0].set_title("make_blobs dataset")
axes[0].set_xlabel("Feature 1")
axes[0].set_ylabel("Feature 2")

# ---- Moons ----
axes[1].scatter(
    X_moons[:, 0],
    X_moons[:, 1],
    c=y_moons,
    cmap="viridis",
    s=30,
    edgecolor="k"
)
axes[1].set_title("make_moons dataset")
axes[1].set_xlabel("Feature 1")
axes[1].set_ylabel("Feature 2")

plt.tight_layout()
plt.show()
```

If you want separate figures instead of side-by-side, you can just do two `plt.figure()` + `plt.scatter(...)` blocks, one for blobs and one for moons.

↳ fig, axes = plt.subplots(1, 2, figsize=(10, 4))

what does the figsize do

`figsize` controls **how big the entire figure window is**.

In this line:

python Copy code

```python
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

- `figsize=(10, 4)` means:
  - **10 inches wide**
  - **4 inches tall**

A few key points:

- It affects the **overall size** of the figure, not individual subplots directly (the subplots share that space).
- The **actual pixel resolution** is `figsize * dpi`.
  - For example, if `dpi=100`, then `(10, 4)` → `1000 x 400` pixels.
- Making the width larger (first number) gives more horizontal room (nice for side-by-

→ fig, axes = plt.subplots(1, 2, figsize=(10, 4))

what does the figsize do

`figsize` controls **how big the entire figure window is**.

In this line:

```python
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

- `figsize=(10, 4)` means:
  - **10 inches wide**
  - **4 inches tall**

A few key points:

- It affects the **overall size** of the figure, not individual subplots directly (the subplots share that space).
- The **actual pixel resolution** is `figsize * dpi`.
  - For example, if `dpi=100`, then `(10, 4)` → `1000 x 400` pixels.
- Making the width larger (first number) gives more horizontal room (nice for side-by-side plots).
- Making the height larger (second number) gives more vertical room (good for tall plots or bigger fonts).

So here, `(10, 4)` is chosen to make a **wide, short** figure that fits two plots side by side comfortably.

what would you write for the kmeans_em.py

Thought for 33s ›

Here's what I'd put in a `kmeans_em.py` file: a self-contained script that

- generates blobs & moons
- runs K-Means and GMM/EM for several k values
- computes silhouette / inertia / log-likelihood / BIC
- saves cluster plots + elbow/BIC plots into a `plots/` folder

You can trim or expand it to match whatever your instructor is asking for.

```python
#!/usr/bin/env python3
"""
kmeans_em.py

Runs K-Means and Gaussian Mixture (EM) on synthetic datasets (blobs and moons),
produces metrics, and saves plots.
"""

import os

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score


RANDOM_STATE = 42
PLOT_DIR = "plots"


# ---------------------------------------------------------------------
# Data generation
# ---------------------------------------------------------------------

def generate_datasets():
    """Generate blobs and moons datasets and return as a dict."""
    X_blobs, y_blobs = make_blobs(
        n_samples=150,
        centers=3,
        cluster_std=1.0,
        random_state=RANDOM_STATE,
    )

    X_moons, y_moons = make_moons(
        n_samples=150,
        noise=0.05,
```

Copy code

```python
def run_kmeans(X, k_values, inits, random_state=RANDOM_STATE):
    """
    Run K-Means for each (k, init) pair.

    Returns a list of dicts with keys:
        dataset, k, init, labels, centers, inertia, silhouette, n_iter
    """
    results = []

    for k in k_values:
        for init in inits:
            km = KMeans(
                n_clusters=k,
                init=init,
                n_init=10,
                random_state=random_state,
            )
            labels = km.fit_predict(X)

            sil = silhouette_score(X, labels)
            inertia = km.inertia_
            n_iter = km.n_iter_

            results.append({
                "k": k,
                "init": init,
                "labels": labels,
                "centers": km.cluster_centers_,
                "inertia": inertia,
                "silhouette": sil,
                "n_iter": n_iter,
            })

    return results


# ---------------------------------------------------------------------
# Gaussian Mixture / EM
# ---------------------------------------------------------------------

def run_gmm(X, k_values, cov_types, random_state=RANDOM_STATE):
    """
    Run GaussianMixture (EM) for each (k, cov_type) pair.

    Returns a list of dicts with keys:
        k, cov_type, labels, silhouette, avg_log_like, bic, model
    """
    results = []

    for k in k_values:
        for cov_type in cov_types:
            gm = GaussianMixture(
                n_components=k,
                covariance_type=cov_type,
                random_state=random_state,
            )
            gm.fit(X)
            labels = gm.predict(X)

            sil = silhouette_score(X, labels)
            avg_log_like = gm.score(X)    # average log-likelihood per sample
            bic = gm.bic(X)

            results.append({
                "k": k,
                "cov_type": cov_type,
                "labels": labels,
                "silhouette": sil,
                "avg_log_like": avg_log_like,
                "bic": bic,
                "model": gm,
            })

    return results


# ---------------------------------------------------------------------
# Plotting helpers
# ---------------------------------------------------------------------

def ensure_plot_dir():
    os.makedirs(PLOT_DIR, exist_ok=True)


def plot_clusters(X, labels, centers=None, title="", save_name=None):
    """Scatter plot of clusters; optionally show centers and save to disk."""
    plt.figure(figsize=(5, 4))
    plt.scatter(
        X[:, 0],
        X[:, 1],
        c=labels,
```

```
        X[:, 1],
        c=labels,
        s=20,
        cmap="viridis",
        edgecolor="k",
    )
    if centers is not None:
        plt.scatter(
            centers[:, 0],
            centers[:, 1],
            marker="X",
            s=200,
            linewidth=2,
            edgecolor="k",
            facecolor="none",
        )

    plt.title(title)
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.tight_layout()

    if save_name is not None:
        path = os.path.join(PLOT_DIR, save_name)
        plt.savefig(path, dpi=150)
        plt.close()
    else:
        plt.show()


def plot_elbow(dataset_name, kmeans_results):
    """
    Plot inertia (sum of squared distances) vs k (elbow plot) for k-means++ init.
    """
    ks = []
    inertias = []

    for res in kmeans_results:
        if res["init"] == "k-means++":
            ks.append(res["k"])
            inertias.append(res["inertia"])

    # Sort by k just in case
    ks, inertias = zip(*sorted(zip(ks, inertias)))

    plt.figure(figsize=(5, 4))
    plt.plot(ks, inertias, marker="o")
    plt.title(f"K-Means Elbow Plot – {dataset_name}")
    plt.xlabel("Number of clusters k")
    plt.ylabel("Inertia (sum of squared distances)")
    plt.tight_layout()

    save_name = f"{dataset_name}_kmeans_elbow.png"
    path = os.path.join(PLOT_DIR, save_name)
    plt.savefig(path, dpi=150)
    plt.close()


def plot_bic(dataset_name, gmm_results):
    """
    Plot BIC vs k for each covariance type.
    """
    cov_types = sorted(set(res["cov_type"] for res in gmm_results))

    plt.figure(figsize=(6, 4))
    for cov_type in cov_types:
        ks = []
        bics = []
        for res in gmm_results:
            if res["cov_type"] == cov_type:
                ks.append(res["k"])
                bics.append(res["bic"])
        ks, bics = zip(*sorted(zip(ks, bics)))
        plt.plot(ks, bics, marker="o", label=cov_type)

    plt.title(f"GMM BIC vs k – {dataset_name}")
    plt.xlabel("Number of components k")
    plt.ylabel("BIC")
    plt.legend()
    plt.tight_layout()

    save_name = f"{dataset_name}_gmm_bic.png"
    path = os.path.join(PLOT_DIR, save_name)
    plt.savefig(path, dpi=150)
    plt.close()


# ---------------------------------------------------------------------
# Main experiment driver
# ---------------------------------------------------------------------
```

```python
# ---------------------------------------------------------------------
# Main experiment driver
# ---------------------------------------------------------------------

def main():
    ensure_plot_dir()

    datasets = generate_datasets()
    k_values = [2, 3, 4, 5]
    kmeans_inits = ["random", "k-means++"]
    gmm_cov_types = ["full", "diag"]

    for name, (X, y_true) in datasets.items():
        print("=" * 70)
        print(f"Dataset: {name}")
        print("=" * 70)

        # -------------------- K-Means --------------------
        print("\nK-Means results:")
        kmeans_results = run_kmeans(X, k_values, kmeans_inits)

        # Print a small summary table
        for res in sorted(kmeans_results, key=lambda r: (r["k"], r["init"])):
            print(f"k={res['k']:>2}, init={res['init']:<10}, "
                  f"silhouette={res['silhouette']:.3f}, "
                  f"inertia={res['inertia']:.1f}, "
                  f"n_iter={res['n_iter']}")

        # Plot K-Means clusterings
        for res in kmeans_results:
            title = f"K-Means – {name} (k={res['k']}, init={res['init']})"
            save_name = f"{name}_kmeans_k{res['k']}_{res['init'].replace('+','plu
            plot_clusters(
                X,
                labels=res["labels"],
                centers=res["centers"],
                title=title,
                save_name=save_name,
            )

        # Elbow plot
        plot_elbow(name, kmeans_results)

        # -------------------- GMM / EM --------------------
        print("\nGMM / EM results:")
        gmm_results = run_gmm(X, k_values, gmm_cov_types)

        for res in sorted(gmm_results, key=lambda r: (r["k"], r["cov_type"])):
            print(f"k={res['k']:>2}, cov_type={res['cov_type']:<4}, "
                  f"silhouette={res['silhouette']:.3f}, "
                  f"avg_log_like={res['avg_log_like']:.3f}, "
                  f"BIC={res['bic']:.1f}")

        # Plot GMM clusterings
        for res in gmm_results:
            title = f"GMM – {name} (k={res['k']}, cov={res['cov_type']})"
            save_name = f"{name}_gmm_k{res['k']}_{res['cov_type']}.png"
            plot_clusters(
                X,
                labels=res["labels"],
                centers=None,  # you could add means here if you want
                title=title,
                save_name=save_name,
            )

        # BIC vs k plot
        plot_bic(name, gmm_results)

    print("\nAll done. Plots saved to:", os.path.abspath(PLOT_DIR))


if __name__ == "__main__":
    main()
```

You can drop this straight into `kmeans_em.py` and run:

```bash
python kmeans_em.py
```

If you want, I can next help you:

- add Gaussian ellipses to the GMM plots, or
- adjust this to match any exact function names / outputs your project spec requires.

where does this use the matplot lib elipse centers?

Thought for 7s ›

Ah, good catch — **it doesn't, yet.**

In the GMM script I wrote, `plot_clusters` only does a scatter of the points colored by cluster label:

```python
def plot_clusters(X, labels, title="", save_name=None):
    plt.figure(figsize=(5, 4))
    plt.scatter(
        X[:, 0],
        X[:, 1],
        c=labels,
        s=20,
        cmap="viridis",
        edgecolor="k",
    )
    ...
```

I left a comment saying you *could* extend it to draw ellipses for each Gaussian, but I didn't actually add that code.

How to actually use ellipses for the GMM components

Here's how you can modify things so you **do** draw ellipses centered at the Gaussian means:

1. Import `Ellipse`:

```python
from matplotlib.patches import Ellipse
```

2. Add a helper to draw a single ellipse from a covariance matrix:

```python
import numpy as np

def draw_ellipse(position, covariance, ax=None, n_std=2.0, **kwargs):
    """
    Draw an ellipse with a given position and covariance.
    n_std controls the radius (in standard deviations).
    """
    ax = ax or plt.gca()

    # Handle diagonal covariance (1D array)
    if covariance.ndim == 1:
        cov = np.diag(covariance)
    else:
        cov = covariance

    # Eigenvalues & eigenvectors
    vals, vecs = np.linalg.eigh(cov)
    # Sort eigenvalues (largest first)
    order = vals.argsort()[::-1]
    vals, vecs = vals[order], vecs[:, order]

    # Width and height of the ellipse (2 * n_std * sqrt(eigenvalues))
    width, height = 2 * n_std * np.sqrt(vals)

    # Angle in degrees of the first eigenvector
    angle = np.degrees(np.arctan2(vecs[1, 0], vecs[0, 0]))

    ellipse = Ellipse(
        xy=position,
        width=width,
        height=height,
        angle=angle,
        fill=False,
        **kwargs
    )
    ax.add_patch(ellipse)
```

3. Update `plot_clusters` to take the **GMM model** and draw ellipses at the component means:

```python
def plot_gmm_clusters(X, labels, model, title="", save_name=None):
    """
    Scatter plot of GMM cluster assign ↓ ts + covariance ellipses.
    """
    plt.figure(figsize=(5, 4))
```

3. Update `plot_clusters` to take the **GMM model** and draw ellipses at the component means:

python Copy code

```python
def plot_gmm_clusters(X, labels, model, title="", save_name=None):
    """
    Scatter plot of GMM cluster assignments + covariance ellipses.
    """
    plt.figure(figsize=(5, 4))
    ax = plt.gca()

    # Scatter points
    scatter = ax.scatter(
        X[:, 0],
        X[:, 1],
        c=labels,
        s=20,
        cmap="viridis",
        edgecolor="k",
    )

    # Draw an ellipse for each component
    means = model.means_
    covariances = model.covariances_
    cov_type = model.covariance_type

    for i, mean in enumerate(means):
        if cov_type == "full":
            cov = covariances[i]
        elif cov_type == "diag":
            cov = np.diag(covariances[i])
        elif cov_type == "tied":
            cov = covariances
        elif cov_type == "spherical":
            cov = np.eye(means.shape[1]) * covariances[i]
        else:
            raise ValueError(f"Unsupported covariance type: {cov_type}")

        draw_ellipse(mean, cov, ax=ax, n_std=2.0, linewidth=2)

    ax.set_title(title)
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    plt.tight_layout()

    if save_name is not None:
        path = os.path.join(PLOT_DIR, save_name)
        plt.savefig(path, dpi=150)
        plt.close()
    else:
        plt.show()
```

4. When you loop over `gmm_results`, call this instead:

python Copy code

```python
for res in gmm_results:
    title = f"GMM – {name} (k={res['k']}, cov={res['cov_type']})"
    save_name = f"{name}_gmm_k{res['k']}_{res['cov_type']}_ellipses.png"
    plot_gmm_clusters(
        X,
        labels=res["labels"],
        model=res["model"],
        title=title,
        save_name=save_name,
    )
```

Now the **ellipse centers** are just the GMM means (`model.means_`), and the ellipse shapes/orientations come from the covariance matrices. The earlier version of the code never used them — this updated version does.